

In the case of default dictionary of the 'collections' package:

```
from collections import defaultdict
d = defaultdict(lambda: "Not Present") # default dictionary
function
d["a"] = 1
d["b"] = 2
print(type(d))
print(d["a"])
print(d["b"])
print(d["c"])
```

...the output is:

```
<class 'collections.defaultdict'>
1
2
Not Present # Instead of an error message it returns a
customised response
```

Counter

A counter tool is used to provide fast text pattern matching. For example, a frequency analysis of a sentence can be carried out with the 'counter' function of the collections package.

```
from collections import Counter
text = """
    A traditional method of analysing texts is to
    find the contentions of each word so that analysts
    can finally conclude the overall contention of the
    text.
    """
cnt = Counter()
for word in text.split():
    cnt[word] += 1
print(type(cnt))
print(cnt)
```

The output is:

```
<class 'collections.Counter'>
Counter({'of': 3, 'the': 3, 'A': 1, 'traditional': 1,
'method': 1, 'analysing': 1, 'texts': 1, 'is': 1, 'to': 1,
'find': 1, 'contentions': 1, 'each': 1, 'word': 1, 'so': 1,
'that': 1, 'analysts': 1, 'can': 1, 'finally': 1, 'conclude':
1, 'overall': 1, 'contention': 1, 'text.': 1})
```

To load the 'counter' module, it is necessary to call the collections package. As shown here, the counter returns a list of dictionary variables from the class collection 'Counter'.

The most common words

Subsequently, finding the most common word items in the collection is also possible. This can be implemented with the *most_common(n)* method of *Counter()*.

```
import re
words = re.findall(r'\w+', text.lower())
x=Counter(words).most_common(5)
print(x)
The output is:
[('of', 3), ('the', 3), ('a', 1), ('traditional', 1),
('method', 1)]
```

First, call the 'findall()' function of 're' to extract all words from the given string:

```
words = re.findall(r'\w+', text.lower())
```

The output is:

```
['a', 'traditional', 'method', 'of', 'analysing', 'texts',
'is', 'to', 'find', 'the', 'contentions', 'of', 'each', 'word',
'so', 'that', 'analysts', 'can', 'finally', 'conclude', 'the',
'overall', 'contention', 'of', 'the', 'text']
```

To eliminate duplicity, convert all strings to lower case. Then, from the word list, it is easy to create a dictionary of unique words using 'Counter()'.

```
x=Counter(words)
print(x)
```

The output is:

```
Counter({'of': 3, 'the': 3, 'a': 1, 'traditional': 1,
'method': 1, 'analysing': 1, 'texts': 1, 'is': 1, 'to': 1,
'find': 1, 'contentions': 1, 'each': 1, 'word': 1, 'so': 1,
'that': 1, 'analysts': 1, 'can': 1, 'finally': 1, 'conclude':
1, 'overall': 1, 'contention': 1, 'text': 1})
```

Text processing is the preamble of natural language processing. The 'collections' package is a powerful tool to initiate text processing. Moreover, it provides alternatives to iterable datatypes like lists, dictionaries, and strings. It also offers additional functionalities for these data types. Some of them have been discussed here with examples. Interested readers can explore this package further in relevant web documents. **END** 🐧

By: Dr Dipankar Ray

The author has more than 15 years of experience in software development. He has published two books on MATLAB and medical image processing (published by PHI Learning and SPD).